

PROYECTO INTEGRADOR M2 – RESUMEN TÉCNICO

Estudiante: Diana Coral, Grupo: dstp10, Academia Henry

I. Primer Avance: Estructura de Tablas y Cargue de Datos

- Script para generar las tablas en PostgreSQL ver archivo [0 Tablas](#).
- Notebook con código corregido para generación de datos con Python, ver archivo [Avance 1 DataGeneration](#) o el Script [01_data_generation.py](#)
- Resumen de la ejecución del llenado de datos ver archivo [generation_summary](#).

1. Estructura y relaciones entre Tablas:

Se tiene una empresa de transporte y logística llamada FleetLogix que opera con 200 vehículos y se carga información de su operación en 6 tablas diferentes descritas a continuación:

1.1. Descripción

- **Tabla drivers:** información de los conductores de la empresa. PK: **driver_id**
- **Tabla routes:** rutas predefinidas entre ciudades. PK: **route_id**
- **Tabla vehicles:** información de los vehículos de la empresa. PK: **vehicle_id**
- **Tabla maintenance:** historial de mantenimiento de los vehículos. PK: **vehicle_id** y **maintenance_id**
- **Tabla trips:** Es el registro de los viajes realizados. PK: **trip_id**
- **Tabla deliveries:** Contiene información de las entregas. PK: **delivery_id**

1.2. Llaves primarias y foráneas

Todas las tablas tienen claves primarias de cuatro dígitos con restricción NOT NULL, es decir que no pueden quedar en blanco o ser NULL porque su información es obligatoria por la relevancia de los datos. Las claves foráneas se organizaron en la siguiente tabla:

Tabla Origen (FK)	Campo FK	Comentario
trips	vehicle_id	Un vehículo puede hacer varios viajes
trips	driver_id	Un conductor puede hacer varios viajes
trips	route_id	Una ruta puede tener varios viajes
deliveries	trip_id	Un viaje tiene varias entregas
maintenance	vehicle_id	Un vehículo puede tener varios mantenimientos

1.3. Constraints

Aparte de los id de cada tabla, otros datos con restricción NOT NULL se listan a continuación:

- **drivers:** employee_code, first_name, last_name, license_number
- **routes:** route_code, origin_city, destination_city
- **vehicles:** license_plate, vehicle_type
- **trips:** departure_datetime
- **deliveries:** tracking_number, customer_name, delivery_address, scheduled_datetime, delivered_datetime
- **maintenance:** maintenance_date, maintenance_type

Para evitar confusiones en la asignación de rutas y entregas, se usa la restricción UNIQUE para:

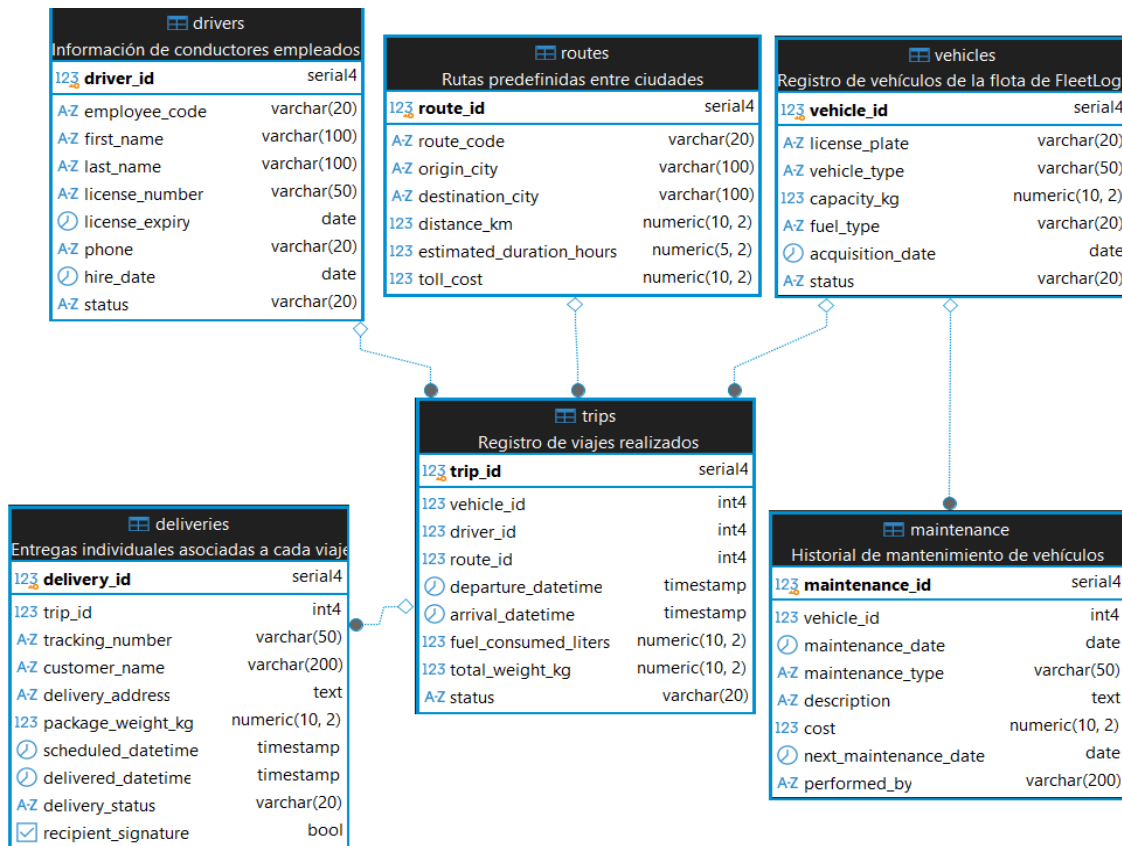
- **drivers.employee_code:** Evita duplicar un mismo empleado y asignarle más de una ruta
- **drivers.license_number:** Evita errores de información de quien se encuentra recorriendo una ruta
- **routes.route_code:** Evita tener distintos destinos con el mismo código
- **vehicles.license_plate:** Evita errores de información del vehículo que recorre una ruta
- **deliveries.tracking_number:** Evita entregar dos paquetes por equivocación a un cliente

1.4. Patrones del negocio:

Las llaves foráneas no permiten eliminar cualquier tabla, por tanto, las tablas maintenance y deliveries que se alimentan de otras tablas pueden ser borradas de primero, seguido de la tabla trips y por último las tablas drivers, routes y vehículos. La tabla trips implícitamente genera la lógica de que cada viaje realizado por la empresa tiene un solo vehículo, un solo conductor y una sola ruta definida. Esto es posible gracias a las llaves foráneas. Además, un viaje se aprovecha para realizar varias entregas según la relación entre deliveries y trips. Por otro lado, se entiende que un vehículo tiene dos historiales de información completamente diferentes, uno es la relación con los viajes y otra con el mantenimiento y se sobreentiende que las rutas ya están predefinidas, pues no incluyen los desvíos por las entregas.

1.5. Diagrama ER

El diagrama ER se ejecutó directamente en la herramienta DBeaver con conexión a Postgres.



1.6. Mejoras

- La tabla vehicles tiene dos historiales independientes: maintenance y trips, pero no hay establecido un límite de capacidad de carga, hecho que pudiese generar consecuencias negativas desde el daño del vehículo, accidentes y multas. El script puede ser mejorado haciendo una función suma del total de peso de las entregas y generando un CHECK para que este sea menor al límite de carga.
- Se debe hacer un CHECK para que un viaje no sea asignado a un conductor cuya licencia está vencida.
- El negocio hace mantenimiento por fecha y no por kilometraje. Sería correcto realizarlo por el segundo factor dependiendo de las políticas de la empresa y las leyes del país.
- Si bien, los viajes son asignados a un conductor, una ruta y un vehículo único, pero se puede traslapar con otro viaje en el mismo horario. Aquí se requiere un CHECK también.
- También se debe considerar la fecha de compra del vehículo ya que un auto que ha cumplido su vida útil no puede seguir siendo usado y se debe hacer su disposición a tiempo. Un CHECK con límite de tiempo de uso podría ayudar en este caso.

2. Cargue de Datos:

Se hizo uso de la plantilla otorgada por la Academia Henry, la cual permite desde la programación en Python el llenado de datos de las tablas disponibles en Postgres. Esta plantilla se explica a continuación.

1. Se llama las librerías necesarias y se establece la conexión con Posgres, En este paso fue necesario colocar el usuario y la clave de Postgres.
2. Se inicializa el generador de datos en idioma español, luego se genera la clase DataGenerator y se reconocen los nombres de las tablas en Postgres.
3. En la clase hay programadas funciones para llenar cada una de las seis tablas que se encuentran en Postgres. Tres de las funciones estaban incompletas #TO DO#: generate_vehicles (query), generate_deliveries (self.cursor.execute) y generate_maintenance (self.cursor.execute).

Python generates	Datos	%	Observaciones			
			Tipo	Capacidad kg	Combustible	Fecha adquisición
vehicles	200	30	Camión Grande	5000	Diesel	Entre 5años y 5 días antes de la fecha actual
		30	Camión Mediano	3000	Diesel	
		30	Van	1500	Gasolina	
		10	Motocicleta	50	Gasolina	
El status genera 90% de los vehículos activos y 10% en mantenimiento						
		Fake*	Licencia/Teléfono	Vencimiento Licencia	Fecha contratación	
drivers	400	Nombre Apellido	Aleatorio entre 1000000000 y 9999999999	Entre 3 años y 1 mes a la fecha actual	Entre 5 años y una semana a la fecha actual	
El status genera 95% de conductores como activos y 5% inactivos						
		Tiempo salida	Tiempo llegada	Combustible	Peso	
trips	100k	Aleatorio últimos dos años	Aproximación con distancia	8-15L /100km	Entre 40% y 90% de la capacidad	
El status genera 95% de viajes ejecutados y 5% por ejecutar						
		Ciudades	Distancia	Duración	Costo total	
routes	50	Bogotá Medellín Cali Barranquilla Cartagena	Usa función get_distance con 10 distancias aproximadas entre ciudades	Velocidades entre 60km/h y 80km/h	Calculo a partir de considerar peaje a \$15000/100km	
		Fake	Peso		Horarios	
deliveries	400k	Nombre Cliente Dirección	Se distribuye de 2 a 6 pedidos		Se hace un aproximado con distancia	
El status genera 90% entregados a tiempo y 10% retraso. La firma genera 95% true y 5% false						
		Fechas	Tipo	Costo	Descripción	
maintenance	5k	Cada 20 viajes	Cambio aceite	150000	Fija	
			Revisión freno	250000		
			Cambio llantas	450000		
			Mantenimiento	350000		
			Revisión Motor	500000		
			Alineación	180000	Nombres de mecánicos generados con Fake	

*La librería Fake permite crear datos aleatorios

1. Se realiza validación de los datos generados y un informe de los datos creados.
2. Una vez se tiene ejecutado el programa de llenado de datos de Python, se procede a hacer una revisión básica de las tablas en Postgres desde DBeaver usando la función SELECT*FROM nombre_tabla.

II. Segundo Avance: Queries y Optimización

- Script para ejecutar las Queries en PostgreSQL, ver archivo [02_queries_analysis](#)
- Script para ejecutar índices de optimización en PostgreSQL, ver archivo [03_optimization_indexes](#)
- Notebook de Queries con resultados, análisis, recomendaciones y tiempos de ejecución antes y después de agregar los índices de optimización, ver el archivo [Avance 2 SQLAnalysis](#).
- Manual de funcionamiento de cada Query, ver archivo [Manual Consultas SQL.pdf](#).

Academia Henry otorgó 12 queries entre básicas, medias y complejas y 5 índices de optimización que se resumen en las siguientes tablas:

Nivel	N	Query
Baja	1	Contar vehículos por tipo.
	2	Listar conductores con licencia próxima a vencer
	3	Total de viajes por estado
Media	4	Total de entregas por ciudad destino en los últimos 2 meses
	5	Conductores activos con cantidad de viajes completados
	6	Promedio de entregas por conductor en los últimos 6 meses
	7	Rutas con mayor consumo de combustible por kilómetro
	8	Análisis de entregas retrasadas por día de la semana
Alta	9	Costo de mantenimiento por kilómetro recorrido
	10	Ranking de conductores por eficiencia usando Window Functions
	11	Análisis de tendencia de viajes con LAG y LEAD
	12	Pivot de entregas por hora y día de la semana

Índice	Tabla	Mejora	Queries
idx_trips_composite_joins	trips	JOINS frecuentes	4,5,6,7,9,10,11
idx_deliveries_scheduled_datetime	deliveries	Análisis temporal	4,8,12
idx_maintenance_vehicle_cost	maintenance	Consultas de costo por vehículo	9
idx_drivers_status_license	drivers	Filtrado de conductores activos	2,5,6,10
idx_routes_metrics	routes	Métricas de eficiencia por ruta	4,7,9,10

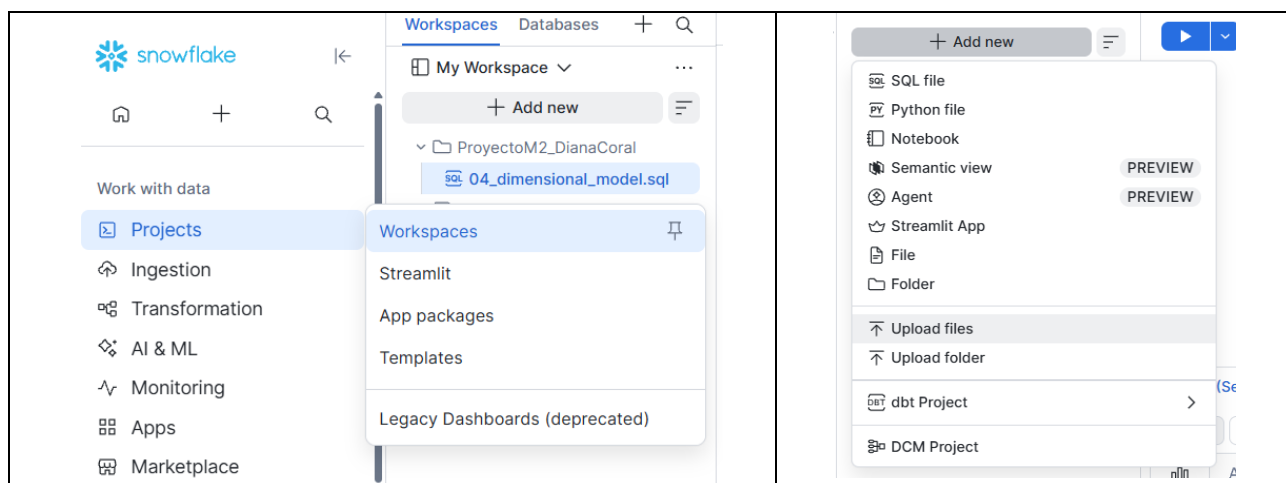
III. Tercer Avance: Data Warehouse en Snowflake y ETL

- Script ya corregido para crear el Warehouse en Snowflake, ver archivo [04_dimensional_model](#)
- Script ya corregido para realizar proceso ETL en Python, ver archivo [05_etl_pipeline](#)
- Notebook sobre código de modelado dimensional y ETL, ver archivo [Avance 3 DataWarehouse](#)
- Ejemplo de la extracción de datos realizada, ver archivo [ejem_extracción_Snowflake](#)

1. Data Warehouse en Snowflake

En el tercer avance se realiza la apertura de la cuenta en Snowflake y se procede a generar el proyecto para generar el Data Warehouse como se indica a continuación:

- a. Dar clic en Projects seguido de Workspaces y Add new. En la nueva pestaña dar clic en Upload files y elegir el archivo 04_dimensional_model.



- b. Una vez se tenga el archivo cargado se quitó las comillas del tamaño del Warehouse 'XSMALL' para que funcione sin errores.
- c. Después de la creación de la base de datos, el código crea las tablas de dimensiones, que son las siguientes:

Dimensión	Descripción	Uso
dim_date	Permite granularidad en diferentes intervalos de tiempo: semestres, trimestres, fines de semana...	Sirve para análisis disgregados por días, meses o años.
dim_time	Permite granularidad a un nivel de: horas, minutos y segundos.	Se analizan tiempo de entrega en minutos, tiempo de retraso, entregas por hora.
dim_vehicle	Contiene los atributos del vehículo.	Identificar métricas relacionadas al vehículo como eficiencia de la gasolina
dim_driver	Contiene los atributos del conductor	Identificar métricas relacionadas al conductor como entregas realizadas por este
dim_route	Contiene los atributos de la ruta	Identificar métricas relacionadas con la ruta como gasolina consumida por destino
dim_customer	Contiene los atributos del cliente	Relación directa con indicadores como estado de entrega de un paquete y firma.

- d. Después de crear las dimensiones, en el código se encuentra la creación de la tabla de hechos fact_deliveries, que se encuentra conformada por:

Hecho	Métricas	Clave → Tabla o Descripción
Llaves	delivery_key date_key scheduled_time_key delivered_time_key vehicle_key driver_key route_key cusstomer_key	PK: fact_deliveries FK: dim_date FK: dim_time FK: dim_time FK: dim_vehicle FK: dim_driver FK: dim_route FK: dim_customer
Dimensiones degeneradas	delivery_id trip_id tracking_number	No tienen más atributos que sus respectivos códigos como por ejemplo la referencia del pedido

Métricas	package_weight_kg distance_km fuel_consumed_liters delivery_time_minutes delay_minutes	Se puede obtener el promedio, el total si se suma y realizar control de límites. Por ejemplo, la cantidad de combustible consumido por la empresa.
Métricas calculadas	deliveries_per_hour fuel_efficiency_km_per_liter cost_per_delivery revenue_per_delivery	Son métricas que se obtienen con operaciones matemáticas previamente realizadas. Por ejemplo, posible ganancia por entrega.
Indicadores	is_on_time is_damage has_signature delivery_status	Son booleanos que permiten conocer el estado de alguna asignación. Por ejemplo, el estado de la entrega, si fue o no entregado.

Al final también tiene un código de Auditoría con `etl_batch_id` y `etl_timestamp`, que permiten rastrear errores de ejecución, generando un código id a cada ejecución del pipeline, de generarse un error se puede eliminar el lote de datos sin afectar el resto. También genera la fecha y hora exacta en que la información fue insertada.

- e. Se habilita la recepción de datos en Snowflake, con un tiempo de 30 días, es decir que la copia estará disponible para analizar esos datos en ese tiempo. Con Staging se genera la tabla de recepción de datos y se registra el momento exacto de su carga.
- f. Finalmente se crea el control del acceso a la información con SECURE VIEW, para dos roles: el analista de ventas y el analista de operaciones. El primero tiene acceso a métricas comerciales como el peso del paquete, es estado de entrega y los ingresos por envío. Mientras que el segundo, no puede observar datos de facturación e ingresos.

2. ETL en Python

- a. Se instala en Python las librerías necesarias para la conexión entre Python, Postgres y Snowflake.
- b. El Código en Python tenía partes sin completar marcadas con `#TO DO#`:
 - o La primera se encontró en el proceso de extracción de datos, en la función `extract_daily_data` y se completó con código para especificar cual columna y de que tabla del modelo OLTP del proyecto FleetLogix en Postgres se debe tomar la información. Además, se hizo un cambio en las condiciones de extracción de información, debido a que los datos de la empresa se generaron ficticios hace ya un mes y no hay más datos nuevos, hecho que dificultaría comprobar la ejecución del pipeline para datos del día anterior, de ahí que se tomó los últimos 46 días (`CURRENT_DATE - INTERVAL '46 days'`). De esta forma se obtuvieron 42 datos extraídos.
 - o También fue necesario realizar un cambio en Snowflake, debido a que no recepcionaba la información, cambiando `customer_key INT PRIMARY KEY` por `customer_key AUTOINCREMENT START 1 INCREMENT 1 PRIMARY KEY`, ya que se generaban null y al ser una llave debe ser un número incremental.
 - o La segunda se encontró en la función `_calculate_daily_totals`, y fue necesario crear código para la creación de la tabla de totales, `daily_delivery_totals` la cual tendrá los resultados de las métricas.
 - o La tercera realiza las operaciones de suma y promedio de las métricas con los datos previamente insertados en `fact_deliveries`.
- c. Se ejecuta el código.

3. Star Schema

Para entender mejor la relación entre la tabla de hechos y las tablas de dimensiones se realizó un esquema de estrella que permite observar las conexiones entre estas y todas sus variables.

dim_date	
123 date_key	int4 NOT NULL
🕒 full_date	date NOT NULL
123 day_of_week	int4
A-Z day_name	varchar(10)
123 day_of_month	int4
123 day_of_year	int4
123 week_of_year	int4
123 month_num	int4
A-Z month_name	varchar(10)
123 quarter	int4
123 year	int4
☒ is_weekend	bool
☒ is_holiday	bool
A-Z holiday_name	varchar(50)
123 fiscal_quarter	int4
123 fiscal_year	int4

dim_time	
123 time_key	int4 NOT NULL
123 hour	int4
123 minute	int4
123 second	int4
A-Z time_of_day	varchar(20)
A-Z hour_24	varchar(5)
A-Z hour_12	varchar(8)
A-Z am_pm	varchar(2)
☒ is_business_hour	bool
A-Z shift	varchar(20)

dim_vehicle	
123 vehicle_key	int4 NOT NULL
123 vehicle_id	int4 NOT NULL
A-Z license_plate	varchar(20)
A-Z vehicle_type	varchar(50)
123 capacity_kg	numeric(10, 2)
A-Z fuel_type	varchar(20)
🕒 acquisition_date	date
123 age_months	int4
A-Z status	varchar(20)
🕒 last_maintenance_date	date
🕒 valid_from	date
🕒 valid_to	date
☒ is_current	bool

dim_customer	
123 customer_key	int4 NOT NULL
123 customer_id	int4
A-Z customer_name	varchar(200)
A-Z customer_type	varchar(50)
A-Z city	varchar(100)
🕒 first_delivery_date	date
123 total_deliveries	int4
A-Z customer_category	varchar(20)

fact_deliveries	
123 delivery_key	int4 NOT NULL
123 date_key	int4
123 scheduled_time_key	int4
123 delivered_time_key	int4
123 vehicle_key	int4
123 driver_key	int4
123 route_key	int4
123 customer_key	int4
123 delivery_id	int4 NOT NULL
123 trip_id	int4 NOT NULL
A-Z tracking_number	varchar(50)
123 package_weight_kg	numeric(10, 2)
123 distance_km	numeric(10, 2)
123 fuel_consumed_liters	numeric(10, 2)
123 delivery_time_minutes	int4
123 delay_minutes	int4
123 deliveries_per_hour	numeric(5, 2)
123 fuel_efficiency_km_per_liter	numeric(5, 2)
123 cost_per_delivery	numeric(10, 2)
123 revenue_per_delivery	numeric(10, 2)
☒ is_on_time	bool
☒ is_damaged	bool
☒ has_signature	bool
A-Z delivery_status	varchar(20)

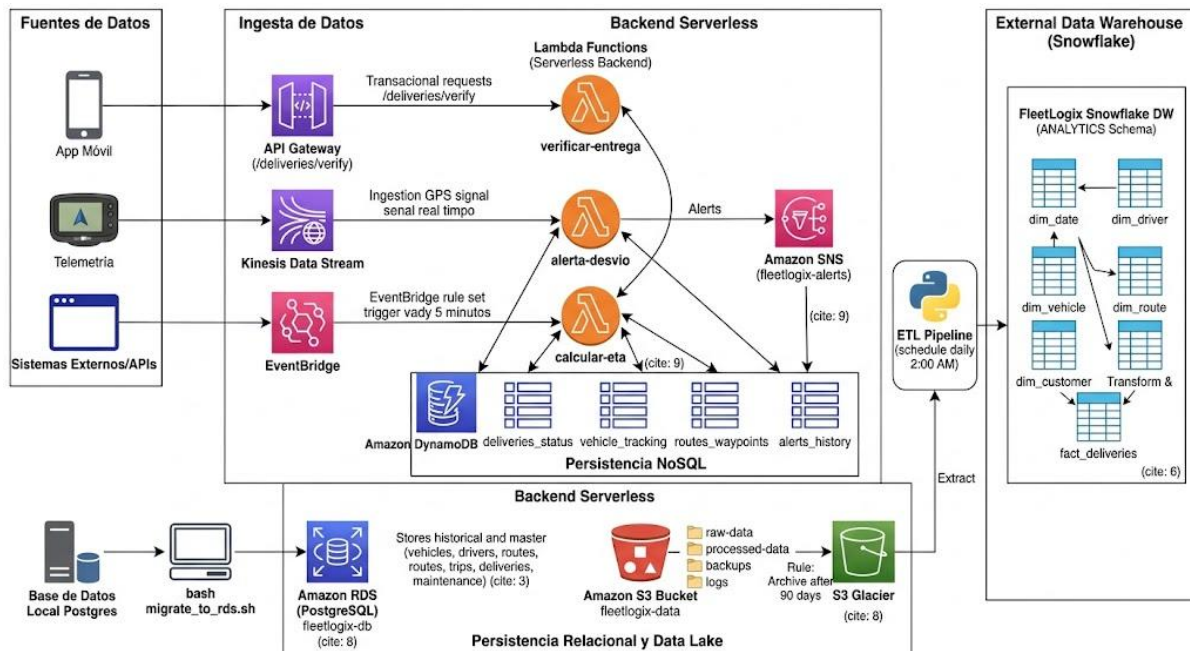
dim_driver	
123 driver_key	int4 NOT NULL
123 driver_id	int4 NOT NULL
A-Z employee_code	varchar(20)
A-Z full_name	varchar(200)
A-Z license_number	varchar(50)
🕒 license_expiry	date
A-Z phone	varchar(20)
🕒 hire_date	date
123 experience_months	int4
A-Z status	varchar(20)
A-Z performance_category	varchar(20)
🕒 valid_from	date
🕒 valid_to	date
☒ is_current	bool

dim_route	
123 route_key	int4 NOT NULL
123 route_id	int4 NOT NULL
A-Z route_code	varchar(20)
A-Z origin_city	varchar(100)
A-Z destination_city	varchar(100)
123 distance_km	numeric(10, 2)
123 estimated_duration_hours	numeric(5, 2)
123 toll_cost	numeric(10, 2)
A-Z difficulty_level	varchar(20)
A-Z route_type	varchar(20)

IV. Cuarto Avance: Arquitectura en la Nube con AWS

- Diagrama con Arquitectura AWS del proyecto FleetLogix, ver archivo [aws_architecture_diagram](#)
- Script de funciones Lambda, ver archivo [06-lambda_functions](#)
- Script de configuración de AWS, ver archivo [07_aws_setup](#)
- Notebook sobre funciones Lambda y configuración de AWS [Avance 4 CloudArchitecture](#)
- Arquitectura técnica, ver archivo [Arquitectura Tecnica](#)
- Diccionario de datos en Excel para modelo OLTP y OLAP, ver archivo [data_dictionary](#)

1. Diagrama Arquitectura AWS para el proyecto FleetLogix



La lógica del diagrama se encuentra explicada en el archivo de Arquitectura Técnica.

2. Scripts Lambda y AWS

Este módulo está conformado por dos Scripts, el primero que genera funciones Lambda y el segundo que permite la creación de la base de datos en AWS. El primero, contiene tres funciones Lambda para realizar la verificación de entregas, calcular el tiempo estimado de llegada de una entrega y posibles alertas de desvío de la ruta. El segundo, contiene 6 funciones para: creación de base de datos en AWS, Creación de Bucket S3, Creación de tablas en DynamoDB, Creación de Buckups automáticos en RDS, Migración a PostgreSQL local a RDS, Creación de funciones Lambda y la función que permite la ejecución completa. Cada uno de estos se explica muy bien en el Notebook 4.

3. Recomendaciones

Los Scripts sugeridos por la academia Henry dejan por defecto el acceso absoluto a toda la información del negocio FleetLogix. Si fuese una empresa con operación real en el mundo comercial, se debe restringir el acceso a datos ya que esto puede generar posibles inconvenientes si personas mal intencionadas acceden a la información. Ejemplo: El conocimiento de las rutas de los vehículos en tiempo real y la predicción de su posición, pueden colocar en peligro la mercancía y al personal, pues si un grupo de delincuentes desea asaltar el vehículo y conocen la información, podrían planificar el robo en partes de la vía con poca vigilancia.